

# Практическое занятие 18: Создание приложения `events` и базовая структура сайта

## Цель занятия:

Создать новое приложение `events` внутри проекта Django, настроить его интеграцию с проектом, разработать базовые маршруты и представления, а также создать и подключить шаблоны для отображения страниц. Дополнительно выполнить задания по созданию приложений `about` и `contact`.

---

## Задачи занятия

1. Создание приложения `events`
  2. Регистрация приложения в проекте
  3. Настройка маршрутов приложения
  4. Создание представлений и шаблонов для `events`
  5. Выполнение практического упражнения. Изменение сообщения на домашней странице
  6. Выполнение дополнительных заданий. Создание приложений `about` и `contact`
- 

## Задача 1: Создание приложения `events`

### 1.1. Открытие проекта в Visual Studio Code (VSCode)

1. Запустите VSCode:
  - **Русский:** Запустите Visual Studio Code.
  - **English:** Open Visual Studio Code.
2. Откройте папку проекта:
  - **Русский:** Перейдите в меню **Файл > Открыть папку...**
  - **English:** Go to **File > Open Folder...**
  - Выберите директорию вашего проекта `Django_Project_OVR/` и нажмите **Открыть (Open)**.

### 1.2. Создание приложения `events` :

1. Откройте терминал в VSCode:

- **Русский:** Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+`` .
- **English:** Go to **View > Terminal** or use the shortcut `Ctrl+`` .

## 2. Выполните команду создания приложения:

```
python manage.py startapp events
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3` .

## 3. Обзор структуры приложения:

После выполнения команды структура проекта будет выглядеть следующим образом:

```
Django_Project_OVR/
|
├─ event_manager/
|   └─ manage.py
|   └─ event_manager/
|       └─ __init__.py
|       └─ settings.py
|       └─ urls.py
|       └─ asgi.py
|       └─ wsgi.py
|   └─ events/
|       └─ __init__.py
|       └─ admin.py
|       └─ apps.py
|       └─ models.py
|       └─ tests.py
|       └─ views.py
|       └─ urls.py
|   └─ venv/
```

**Самостоятельно в ходе реализации практического задания вы добавите:**

```
|   └─ templates/
|       └─ events/
|           └─ home.html
|   └─ static/
|       └─ events/
```

```
| | |
| | |   └─ css/
| | |   └─ styles.css
```

---

## Задача 2: Регистрация приложения в проекте

### 1. Откройте файл настроек `settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

### 2. Добавьте приложение `events` в список `INSTALLED_APPS`:

Найдите раздел `INSTALLED_APPS` и добавьте `'events'`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'events', # Добавлено приложение events  
]
```

### 3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.

---

## Задача 3: Настройка маршрутов приложения

### 3.1. Создание файла `urls.py` в приложении `events`:

#### 1. Создайте файл `urls.py`:

- В папке `events/` создайте новый файл с именем `urls.py`.

#### 2. Добавьте маршруты в `events/urls.py`:

Вставьте следующий код:

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.index, name='index'),  
    path('about/', views.about, name='about'),  
]
```

```
path('', views.home, name='home'),  
]
```

### 3.2. Подключение маршрутов приложения к проекту:

#### 1. Откройте файл `event_manager/urls.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `urls.py`.

#### 2. Импортируйте `include` и добавьте маршрут для приложения `events`:

Обновите содержимое файла следующим образом:

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls')), # Подключение маршрутов  
    приложения events  
]
```

#### 3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.

---

## Задача 4: Создание представлений и шаблонов для `events`

### 4.1. Создание функции `home` в `events/views.py`:

#### 1. Откройте файл `events/views.py`:

- В папке `events/` откройте файл `views.py`.

#### 2. Добавьте функцию `home`:

Замените или добавьте следующую функцию:

```
from django.shortcuts import render  
  
def home(request):  
    return render(request, 'events/home.html')
```

#### 3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

## 4.2. Создание шаблона `home.html` :

### 1. Создайте директорию для шаблонов:

- В корневой директории проекта (на одном уровне с `manage.py` ) создайте папку `templates` , если она ещё не существует.
- Внутри `templates/` создайте папку `events` .

### 2. Создайте файл `home.html` :

- В папке `templates/events/` создайте файл `home.html` .

### 3. Добавьте содержимое в `home.html` :

Вставьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
  <title>Главная страница</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Добро пожаловать в систему управления мероприятиями!</h1>
</body>
</html>
```

## 4.3. Создание файла стилей `styles.css` :

### 1. Создайте файл `styles.css` :

- В папке `static/events/css/` создайте файл `styles.css` .

### 2. Добавьте следующий CSS-код для стилизации страницы:

```
body {
  background-color: #f0f0f0;
  font-family: Arial, sans-serif;
  text-align: center;
  padding-top: 50px;
}

h1 {
  color: #333333;
}
```

### 3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

## 4.4. Настройка путей к шаблонам и статическим файлам в `settings.py` :

1. Откройте файл `event_manager/settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

2. Убедитесь, что параметр `'DIRS'` настроен следующим образом:

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

3. Настройка статических файлов:

Убедитесь, что следующие параметры присутствуют и настроены:

```
STATIC_URL = '/static/'  
STATICFILES_DIRS = [BASE_DIR / 'static']
```

4. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

---

## Задача 5: Запуск и проверка работы приложения

1. Активируйте виртуальное окружение:

- На Windows:

```
venv\Scripts\activate
```

- На macOS/Linux:

```
source venv/bin/activate
```

2. Запустите сервер разработки:

```
python manage.py runserver
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3`.

### 3. Проверьте отображение страницы:

- Откройте веб-браузер.
- Перейдите по адресу: <http://127.0.0.1:8000/>
- Вы должны увидеть страницу с заголовком "Добро пожаловать в систему управления мероприятиями!" и применёнными стилями из `styles.css`.

### 4. Остановка сервера разработки:

- В терминале, где запущен сервер, нажмите сочетание клавиш `Ctrl + C` (Windows) или `Cmd + C` (macOS/Linux).

---

## Задача 6: Практическое упражнение

### Изменение сообщения на домашней странице:

#### 1. Откройте файл `home.html`:

- В папке `templates/events/` откройте файл `home.html`.

#### 2. Измените текст в `<h1>`:

- Найдите строку `<h1>Добро пожаловать в систему управления мероприятиями!`  
`</h1>` и измените её на:

```
<h1>Добро пожаловать в нашу новую систему управления мероприятиями!  
</h1>
```

- Добавьте строку:

```
<p>Скоро здесь вы сможете создавать мероприятия и регистрироваться  
на них.</p>
```

#### 3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

#### 4. Запустите сервер и проверьте изменения:

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/> и убедитесь, что сообщение изменилось.

---

# Дополнительные задания: Создание приложений `about` и `contact`

## Цель:

Расширить функциональность проекта, добавив приложения `about` и `contact`, которые будут содержать страницы "О нас" и "Контакты" соответственно.

## Задачи:

1. Создание приложений `about` и `contact`.
  2. Регистрация приложений в проекте.
  3. Настройка маршрутов для новых приложений.
  4. Создание представлений и шаблонов для страниц "О нас" и "Контакты".
  5. Реализация формы обратной связи на странице "Контакты".
- 

## Задание 1: Создание приложения `about`

### 1.1. Создание приложения `about`:

#### 1. Откройте терминал в VSCode:

- **Русский:** Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+``.
- **English:** Go to **View > Terminal** or use the shortcut `Ctrl+``.

#### 2. Выполните команду создания приложения:

```
python manage.py startapp about
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3`.

### 1.2. Регистрация приложения `about` в проекте:

#### 1. Откройте файл `event_manager/settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

#### 2. Добавьте приложение `about` в список `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    ...  
    'events',
```

```
'about', # Добавлено приложение about  
]
```

### 3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

## 1.3. Настройка маршрутов для приложения `about`:

### 1. Создайте файл `urls.py` в приложении `about`:

- В папке `about/` создайте новый файл с именем `urls.py`.

### 2. Добавьте маршруты в `about/urls.py`:

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.about, name='about'),  
]
```

### 3. Подключите маршруты приложения `about` к проекту:

- Откройте файл `event_manager/urls.py`:

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls')),  
    path('about/', include('about.urls')), # Подключение маршрутов  
                                           приложения about  
]
```

### 4. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

## 1.4. Создание представления и шаблона для страницы "О нас":

### 1. Создайте функцию `about` в `about/views.py`:

```
from django.shortcuts import render  
  
def about(request):  
    return render(request, 'about/about.html')
```

### 2. Создайте шаблон `about.html`:

- **Создайте директорию для шаблонов:**
  - В папке `templates/` создайте папку `about`.
- **Создайте файл `about.html` в `templates/about/`:**

```
<!DOCTYPE html>
<html>
<head>
  <title>О нас</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>О нашей компании</h1>
  <p>Информация о компании и команде.</p>
</body>
</html>
```

### 3. Сохраните изменения и проверьте страницу:

- Запустите сервер:

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/about/>
- Убедитесь, что страница "О нас" отображается корректно.

---

## Задание 2: Создание приложения `contact`

### 2.1. Создание приложения `contact`:

#### 1. Выполните команду создания приложения:

```
python manage.py startapp contact
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3`.

### 2.2. Регистрация приложения `contact` в проекте:

#### 1. Откройте файл `event_manager/settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

## 2. Добавьте приложение `contact` в список `INSTALLED_APPS` :

```
INSTALLED_APPS = [  
    ...  
    'events',  
    'about',  
    'contact', # Добавлено приложение contact  
]
```

### 3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

## 2.3. Настройка маршрутов для приложения `contact` :

### 1. Создайте файл `urls.py` в приложении `contact` :

- В папке `contact/` создайте новый файл с именем `urls.py`.

### 2. Добавьте маршруты в `contact/urls.py` :

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.contact, name='contact'),  
]
```

### 3. Подключите маршруты приложения `contact` к проекту:

- Откройте файл `event_manager/urls.py` :

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls')),  
    path('about/', include('about.urls')),  
    path('contact/', include('contact.urls')), # Подключение  
    маршрутов приложения contact  
]
```

### 4. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

## 2.4. Создание представления и шаблона для страницы "Контакты":

### 1. Создайте функцию `contact` в `contact/views.py` :

```

from django.shortcuts import render
from django.http import HttpResponseRedirect

def contact(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        email = request.POST.get('email')
        message = request.POST.get('message')
        # Здесь можно добавить логику обработки формы, например,
        # отправку email
        return HttpResponseRedirect("Спасибо за ваше сообщение!")
    return render(request, 'contact/contact.html')

```

## 2. Создайте шаблон `contact.html`:

- **Создайте директорию для шаблонов:**
  - В папке `templates/` создайте папку `contact`.
- **Создайте файл `contact.html` в `templates/contact/`:**

```

<!DOCTYPE html>
<html>
<head>
    <title>Контакты</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Свяжитесь с нами</h1>
    <form method="post">
        {% csrf_token %}
        <label for="name">Имя:</label>
        <input type="text" id="name" name="name" required><br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required><br>
    <br>

        <label for="message">Сообщение:</label><br>
        <textarea id="message" name="message" rows="5" cols="40"
required></textarea><br><br>

        <button type="submit">Отправить</button>
    </form>
</body>
</html>

```

## 3. Сохраните изменения и проверьте страницу:

- Запустите сервер:

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/contact/>
- Убедитесь, что страница "Контакты" отображается корректно и форма работает.

---

## Дополнительные подсказки по меню и терминалам

### Терминал в Visual Studio Code (VSCode)

Русский:

- **Открытие терминала:**
  - Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+``.
- **Завершение работы терминала:**
  - Нажмите иконку **Trash** (мусорное ведро) в верхней части терминала или перейдите в меню **Терминал > Завершить терминал**.

English:

- **Opening the Terminal:**
  - Go to **View > Terminal** or use the shortcut `Ctrl+``.
- **Killing the Terminal:**
  - Click the **Trash** icon at the top of the terminal or go to **Terminal > Kill Terminal**.

### Интерфейс VSCode

Русский:

- **Панель расширений:** Слева нажмите на иконку **Расширения** или используйте `Ctrl+Shift+X`.
- **Панель поиска файлов:** Используйте сочетание клавиш `Ctrl+P` для быстрого поиска файлов.
- **Панель управления Git:** Нажмите на иконку **Source Control** (иконка с веткой) или используйте `Ctrl+Shift+G`.

English:

- **Extensions Panel:** Click on the **Extensions** icon on the left or use `Ctrl+Shift+X`.

- **File Search Panel:** Use the shortcut `Ctrl+P` to quickly search for files.
- **Git Control Panel:** Click on the **Source Control** icon (branch icon) on the left or use `Ctrl+Shift+G`.

## Общие советы по работе с терминалом в VSCode

- **Навигация по терминалу:**
    - Используйте стрелки вверх и вниз для просмотра предыдущих команд.
  - **Множественные терминалы:**
    - Вы можете открыть несколько терминалов одновременно, нажав на кнопку **+** в верхней части панели терминала.
  - **Переключение между терминалами:**
    - Используйте выпадающее меню рядом с кнопкой **+** для переключения между открытыми терминалами.
- 

## Заключение

Поздравляем! Вы успешно:

- Создали новое приложение `events` внутри проекта Django.
  - Зарегистрировали приложение в настройках проекта.
  - Настроили маршруты и представления для приложения.
  - Создали и подключили шаблоны для отображения страниц.
  - Добавили дополнительные приложения `about` и `contact` с соответствующими страницами и формой обратной связи.
  - Запустили сервер разработки и проверили работу приложения.
- 

**Вы готовы к дальнейшему изучению и разработке на Django.** В следующих практических занятиях мы будем углубляться в создание моделей, работу с базой данных, настройку административного интерфейса и другие аспекты Django.

Если у вас остались вопросы или возникли трудности, пожалуйста, обратитесь за помощью к преподавателю или обсудите с одногруппниками.

**Успехов в разработке!**

---

## Вопросы для обсуждения

- **Почему важно регистрировать приложение в INSTALLED\_APPS?**
  - Регистрация приложения в INSTALLED\_APPS позволяет Django обнаруживать и интегрировать его компоненты, такие как модели, админ-панель, шаблоны и статические файлы. Это необходимо для корректной работы приложения внутри проекта, обеспечения доступа к его функциональности и возможности применять миграции базы данных.
  - **Как работает подключение маршрутов приложения к проекту через функцию include?**
  - Функция include позволяет включать маршруты из приложения в основной файл urls.py проекта. Это обеспечивает модульность и упрощает управление URL-адресами, позволяя каждому приложению определять свои собственные маршруты, которые затем объединяются в общую систему URL проекта. Это особенно полезно при работе с несколькими приложениями в одном проекте.
  - **Как Django находит и использует шаблоны для отображения страниц?**
  - Django использует настройки TEMPLATES в settings.py для определения путей к шаблонам. При вызове функции render или использовании классов-представлений, Django ищет соответствующий шаблон в указанных директориях, включая директории внутри приложений, если включен параметр APP\_DIRS. Затем шаблоны рендерятся с предоставленными контекстными данными и возвращаются как HTTP-ответы пользователю.
- 

## Полезные ресурсы

- [Документация Django](#)
- [Документация Git](#)
- [Django Templates](#)
- [Django Static Files](#)
- [Документация VSCode](#)